fn match_per_group_predicate

Michael Shoemate (@Shoeboxam)

November 13, 2026

> This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `match_per_group_predicate` in `mod.rs` at commit `f5bb719` (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

Types matching pseudocode.

Caller-verified

None

Pseudocode

```
def match_per_group_predicate(
    enumeration: Expr,
    partition_by: Vec[Expr],
    identifier: Expr,
    threshold: u32,
) -> Optional[Bound]:
    range = int_range(lit(0), len(), 1, DataType.Int64)
    if enumeration == range:
        raise "physical row order is not invariant under unordered identifier metrics"

    trusted = match_permute(enumeration) == range
    one_based = False
    if not trusted:
        trusted = match_ordinal_rank(
            enumeration, range
        )  #
        one_based = trusted
    if not trusted:
        return None

    ids, by = partition(
        lambda expr: expr == identifier, partition_by
    )  #
    if not ids:
```

[1] See new changes with `git diff f5bb719..71a5f401a rust/src/transformations/make_stable_lazyframe/truncate/matching/mod.rs`

```
        raise "failed to find identifier column in per_group predicate condition"
    return Bound(
        by=by, per_group=max(0, threshold - int(one_based)), num_groups=None
    )  #


def match_ordinal_rank(enumeration: Expr, range: Expr) -> bool:
    if not (
        isinstance(enumeration, Expr.Function)
        and isinstance(enumeration.function, FunctionExpr.Rank)
        and enumeration.function.options.method == RankMethod.Ordinal
        and len(enumeration.input) == 1
        and isinstance(enumeration.input[0], Expr.Function)
        and enumeration.input[0].function == FunctionExpr.AsStruct
    ):
        return False

    fields = enumeration.input[0].input
    if not fields:
        return False
    *keys, last = fields
    if not isinstance(last, Expr.Alias) or match_permute(last.expr) != range:
        return False

    for key in keys:  #
        if not is_permutation_equivariant(key):
            raise "SortBy truncation keys must be permutation-equivariant"
    return True
```

Postcondition

Theorem 1.1 (Postcondition)**.** If `enumeration` is an enumeration of rows, and `partition_by` includes `identifier`, then returns a `threshold` bound on per-group contribution, when grouped by the non-identifier columns in `partition_by`.

Proof. The matcher accepts only a full trusted uniform permutation of `int_range(0, len)`, or ordinal ranks of a struct consisting of permutation-equivariant sort keys (or invariant scalars) and a final trusted uniform permutation. Line 16 rejects other expressions.

In the sorted case, each row has a distinct random tie-breaking priority. The sort keys commute with input-row permutations and are infallible and non-resizing, as checked on line 49. This condition does not imply that each key is row-local: for example, uniqueness tests can depend on other rows in the partition. The enclosing window evaluates the ranking within each partition, which contains the identifier. Consequently, changing one identifier's rows cannot affect keys in another identifier's partition. The uniform tie-breaker makes selection invariant in distribution under physical input-row permutations.

By line 23, the window partitions rows by identifier and the additional grouping expressions. Full permutations assign distinct values $0, \ldots, n-1$; ordinal ranks assign $1, \ldots, n$. Line 28 accounts for this offset when converting the predicate threshold into a per-group contribution bound. □